CS2323 Computer Architecture

Homework-3: Floating point arithmetic
MA24BTECH11022

Q1. Convert the following decimal numbers into IEEE-754 floating-point format (write the final answer in Hex). Show all steps [4 marks]

(a) -13.25 (single precision)
The integer part is $13_{10} = 1101_2$ and the fractional part is $0.25_{10} = 0.01_2$. So, $-13.25_{10} = -1101.01_2$. After normalizing, $-13.25_{10} = -1.10101_2 \times 2^3$, therefore $S = 1$, and the exponent $e = 3$. For single precision, the bias is 127. So the biased exponent $E = e + 127 = 130_{10} = 10000010_2$.
The fraction bits are $F = 10101000000000000000000$. So, the IEEE-754 representation is

$$1 \mid 10000010 \mid 10101000000000000000000$$

After grouping into groups of 4 bits, the representation is

$$1100\ 0001\ 0101\ 0100\ 0000\ 0000\ 0000\ 0000.$$

So the representation in Hex = `0xC1540000`.

(b) 0.1 (single precision)
The integer part is $0_{10} = 0_2$ and the fractional part is 0.1_{10}. Let us convert the fractional part to binary.
Observe that

$$0.1 \times 2 = 0.2$$
$$0.2 \times 2 = 0.4$$
$$0.4 \times 2 = 0.8$$
$$0.8 \times 2 = 1.6$$
$$0.6 \times 2 = 1.2$$
$$0.2 \times 2 = 0.4$$

The bits repeat, so $0.1_{10} = 0.0\overline{0011}_2$. After normalizing, $0.1_{10} = 1.\overline{1001}_2 \times 2^{-4}$, therefore $S = 0$, and the exponent $e = -4$. For single precision, the bias is 127. So the biased exponent $E = e + 127 = 123_{10} == 01111011_2$.
The first 23 fraction bits are 10011001100110011001100. The next bit is 1 so we round up to get 10011001100110011001101.
The IEEE-754 representation is

$$0 \mid 01111011 \mid 10011001100110011001101$$

After grouping into groups of 4 bits, the representation is

$$0011\ 1101\ 1100\ 1100\ 1100\ 1100\ 1100\ 1101.$$

So the representation in Hex = `0x3DCCCCCD`.

(c) 156.75 (double precision)
The integer part is $156_{10} = 10011100_2$ and the fractional part is $0.75_{10} = 0.11_2$. So, $156.75_{10} = 10011100.11_2$. After normalizing, $156.75_{10} = 1.001110011_2 \times 2^7$, therefore $S = 0$, and the exponent $e = 7$. For double precision, the bias is 1023. So the biased exponent $E = e + 1023 = 1030_{10} = 10000000110_2$.
The fraction bits are $F = 0011100110000000000000000000000000000000000000000000$. So, the IEEE-754 representation is

$$0 \mid 10000000110 \mid 0011100110000000000000000000000000000000000000000000$$

After grouping into groups of 4 bits, the representation is

$$0100\ 0000\ 0110\ 0011\ 1001\ 1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000$$

So the representation in Hex = `0x4063980000000000`.

(d) -0.0078125 (double precision)
The integer part is $0_{10} = 0_2$ and the fractional part is $0.0078125_{10} = 0.0000001_2$. So, $-0.0075125_{10} = -0.0000001_2$. After normalizing, $-0.0078152_{10} = -1.0_2 \times 2^{-7}$, therefore $S = 1$, and the exponent $e = -7$. For double precision, the bias is 1023. So the biased exponent $E = e + 1023 = 1016_{10} = 01111111000_2$.
The fraction bits are $F = 0000000000000000000000000000000000000000000000000000$. So, the IEEE-754 representation is

$$1 \mid 01111111000 \mid 0000000000000000000000000000000000000000000000000000$$

After grouping into groups of 4 bits, the representation is

$$1011\ 1111\ 1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000.$$

So the representation in Hex = `0xBF80000000000000`.

Q2. Convert the following hexadecimal values into their decimal equivalents. Show steps. [6 marks]

(a) `0xC1200000` (single precision)
`0xC1200000` = $11000001001000000000000000000000_2$. By separating into sign, exponent and fraction we have

$$1 \mid 10000010 \mid 01000000000000000000000$$

So $S = 1$ and $E = 10000010_2 = 130_{10}$. For single precision, the actual exponent $e = E - 127 = 3$.
The fraction $F = 01000000000000000000000_2$, so $1.F = 1.01_2 = 1.25_{10}$.
The decimal value is $(-1)^S(1.F)2^e = (-1)^1(1.25)2^3 = -10$.

(b) `0x3F800000` (single precision)
`0x3F800000` = $00111111100000000000000000000000_2$. By separating into sign, exponent and fraction we have

$$0 \mid 01111111 \mid 00000000000000000000000$$

So $S = 0$ and $E = 01111111_2 = 127_{10}$. For single precision, the actual exponent $e = E - 127 = 0$.
The fraction $F = 00000000000000000000000_2$, so $1.F = 1.0_2 = 1_{10}$.
The decimal value is $(-1)^S(1.F)2^e = (-1)^0(1)2^0 = 1$.

(c) `0xBFF0000000000000` (double precision)
`0xBFF0000000000000` = $1011111111110000000000000000000000000000000000000000000000000000_2$. By separating into sign, exponent and fraction we have

$$1 \mid 01111111111 \mid 0000000000000000000000000000000000000000000000000000$$

So $S = 1$ and $E = 01111111111_2 = 1023_{10}$. For double precision, the actual exponent $e = E - 1023 = 0$.
The fraction $F = 0000000000000000000000000000000000000000000000000000_2$, so $1.F = 1_2 = 1_{10}$.
The decimal value is $(-1)^S(1.F)2^e = (-1)^1(1)2^0 = -1$.

(d) `0x4024000000000000` (double precision)
`0x4024000000000000` = $0100000000100100000000000000000000000000000000000000000000000000$. By separating into sign, exponent and fraction we have

$$0 \mid 10000000010 \mid 0100000000000000000000000000000000000000000000000000$$

So $S = 0$ and $E = 10000000010_2 = 1026_{10}$. For double precision, the actual exponent $e = E - 1023 = 3$.
The fraction $F = 0100000000000000000000000000000000000000000000000000_2$, so $1.F = 1.01_2 = 1.25_{10}$.
The decimal value is $(-1)^S(1.F)2^e = (-1)^0(1.25)2^3 = 10$.

Q3. You are given two IEEE-754 single-precision numbers as 32-bit hex values: [4 marks]
A = 0x41480000 (single-precision)
B = 0xC0700000 (single-precision)
Perform the addition A + B and write the final answer in IEEE-754 double precision format.

I shall convert A and B to binary without showing all the steps as before to keep things clean.
A = $1.1001_2 \times 2^3$ and B = $-1.111_2 \times 2^1$. Since B has the smaller exponent, we shift its significand $3-1=2$ places to the right. Now B = $-0.01111_2 \times 2^3$. A+B = $1.1001_2 \times 2^2 + (-0.01111_2 \times 2^3)$.

$$\begin{array}{r} 1.10010_2 \\ -0.01111_2 \\ \hline 1.00011_2 \end{array}$$

Therefore, A+B = $1.00011_2 \times 2^3 = 8.75_{10}$. Now we convert this to double precision format without showing all the steps as we did in Q1 to again keep things clean. There is no need for rounding. The sum is `0x4021800000000000`.

Q4. You are given two IEEE-754 double-precision numbers as 64-bit hex values: [4 marks]
A = 0x4039000000000000 (double precision)
B = 0xC008000000000000 (double precision)
Perform the multiplication A x B and write the final answer in IEEE-754 single precision format.

I shall convert A and B to binary without showing all the steps as before to keep things clean.
A = $1.1001_2 \times 2^4$ and B = $-1.100_2 \times 2^1$. The unbiased exponents are 4 and 1, so the exponent of the product before normalization is 5. We will now compute $1.1001_2 \times 1.100_2$.

$$\begin{array}{r} 1.1001_2 \\ \times 1.100_2 \\ \hline 00000 \\ 00000 \\ 11001 \\ 11001 \\ \hline 100101100_2 \end{array}$$

Therefore, before normalization the product is $-10.0101100_2 \times 2^5$. I shall convert this product to binary without showing all the steps as before to keep things clean. There is again no need for rounding, so the final product is `0xC2960000`.

Q5. Show one example to prove that addition is not associative for floating point numbers i.e., $(a+b)+c \neq a+(b+c)$ [2 marks]

```
#include <iostream>
#include <iomanip>
using namespace std;

int main(){
    float a = 1.0e20f, b = -1.0e20f, c = 1.0f;
    float x = (a+b)+c, y = a+(b+c);

    cout<<setprecision(10);
    cout<<"(a+b)+c = "<<x<<'\n';
    cout<<"a+(b+c) = "<<y<<'\n';
}

Output:
(a+b)+c = 1
a+(b+c) = 0
```

The code demonstrates that addition is not associative for floating point numbers. During the computation of `y`, `-1.0e20f+1.0f` is rounded down to `-1.0e20f`, so we get 0 as the result, whereas in the other case, `1.0e20f-1.0e20f` returns `0` and we get 1 as the final sum.